UNIVERSIDAD DON BOSCO

Escuela de Computación



MANUAL TÉCNICO

Sistema de administración y reserva de habitaciones hoteleras

API REST con Spring Boot y SQL Server

Asignatura: Desarrollo de Aplicaciones con Web Frameworks (DWF404)

Catedrático: Ing. Mario Alvarado

Grupo: G04L

Integrante	Código	Rol en el proyecto
Manuel de Jesús Hernández Escamilla	HE252995	Coordinador y desarrollador
Manuel Enrique Miranda Rivas	MR252985	Desarrollador frontend
Walter Fabricio Salazar Consuegra	SC252970	Desarrollador backend

Fecha de entrega: 29 de octubre de 2026

Versión 1.0

Índice

1. Introducción 3
1.1 Objetivos 3
1.2 Procesos o módulos 3
1.3 Descripción 3
2. Herramientas y tecnologías 3
2.1 Dependencias y versiones 4
3. Arquitectura 4
4. Estructura del proyecto 5
5. Configuración y ejecución 5
5.1 Requisitos 5
5.2 Base de datos 5
5.3 Variables de entorno 6
5.4 Ejecución 6
6. Modelo de datos 6
7. Roles y permisos 8
8. Referencia de los endpoints 8
8.1 Autenticación y perfil 8
8.2 Catálogos 8
8.3 Usuarios y huéspedes 9
8.4 Disponibilidad, reservas y pagos 9
8.5 Reclamos y estadísticas 10
9. Reglas de negocio 10
9.1 Reservas 10
9.5 Tarifas 10
9.2 Pagos 10
9.3 Cancelaciones 10
9.4 Estadísticas 10
10. Formato de errores 11
11. Pruebas del sistema 11
12. Limitaciones y trabajo futuro 11
13. Usuarios y datos de prueba 11
13.1 Datos base 12
14. Repositorio y acceso 12

1. Introducción

Este manual documenta la API REST de la Fase Final del proyecto. Describe los procesos o módulos implementados, las herramientas y versiones utilizadas, la estructura de paquetes, el modelo relacional de la base de datos y los usuarios de prueba para validar el sistema. Está pensado para que cualquier integrante del equipo, el docente o una persona que reciba el proyecto pueda instalarlo y entender cómo funciona.

1.1 Objetivos

- Implementar una API REST con Spring Boot que cubra los procesos de un sistema de reservas de habitaciones de hotel.
- Persistir la información en Microsoft SQL Server con operaciones CRUD y consultas por rango de fechas.
- Controlar el acceso con JWT y roles (ADMIN, RECEPCIONISTA y HUESPED), y validar los datos del lado del servidor.
- Documentar la API con OpenAPI y Swagger UI.

1.2 Procesos o módulos

Módulo	Procesos principales
Autenticación	Login, registro de huéspedes y consulta del usuario autenticado.
Perfil	Consulta de los datos personales del huésped que ha iniciado sesión.
Catálogos	Hoteles, sucursales, tipos de habitación, habitaciones, tarifas, servicios y políticas de cancelación.
Usuarios y huéspedes	Administración de personal y de clientes, con activación y desactivación de cuentas.
Disponibilidad	Búsqueda de habitaciones libres por fechas, capacidad, ciudad y tipo.
Reservas	Creación con titular y huéspedes, cálculo de precio, cambio de fechas, cancelación con reembolso y finalización.
Pagos	Registro de pagos simulados y listado por fecha.
Reclamos	Creación por parte del huésped y atención por parte del personal.
Estadísticas	Reservas por estado, ocupación, ingresos netos y reembolsos por rango de fechas.

1.3 Descripción

API REST para administrar hoteles, sucursales, tipos de habitación, habitaciones, tarifas, servicios adicionales, políticas de cancelación, usuarios, huéspedes, disponibilidad, reservas, pagos, cancelaciones, reclamos y estadísticas. El sistema puede administrar un hotel o una cadena hotelera, porque cada sucursal pertenece a un hotel y cada política de cancelación también. El backend expone la API y el frontend (HTML, CSS y JavaScript) la consume.

2. Herramientas y tecnologías

Componente	Tecnología
Lenguaje	Java 17
Framework	Spring Boot 3.3.5
Persistencia	Spring Data JPA, Hibernate 6.5
Base de datos	Microsoft SQL Server (base HotelReserva)
Seguridad	Spring Security, JWT (jjwt 0.12.6), BCrypt
Validación	Bean Validation (Jakarta)
Documentación de API	springdoc-openapi 2.6.0 (Swagger UI)
Utilidades	Lombok

Componente	Tecnología
Construcción	Maven
Frontend	HTML5, CSS3, JavaScript y Bootstrap 5.3.3
IDE	IntelliJ IDEA
Gestor de base de datos	SQL Server Management Studio (SSMS)
Diseño de prototipos	Figma
Control de versiones	GitHub
Gestión de tareas	Trello

2.1 Dependencias y versiones

Las dependencias están definidas en el archivo pom.xml del backend. Las versiones marcadas como administradas las define el padre spring-boot-starter-parent 3.3.5.

Dependencia	Versión
Java (JDK)	17
Spring Boot (padre Maven)	3.3.5
Spring Framework (spring-webmvc, spring-core)	6.1.14
spring-boot-starter-web	Administrada por Spring Boot 3.3.5
spring-boot-starter-data-jpa	Administrada por Spring Boot 3.3.5
Hibernate ORM	6.5.3
spring-boot-starter-validation	Administrada por Spring Boot 3.3.5
spring-boot-starter-security	Administrada por Spring Boot 3.3.5
mssql-jdbc (driver de SQL Server)	12.6.4
jjwt-api, jjwt-impl, jjwt-jackson	0.12.6
springdoc-openapi-starter-webmvc-ui	2.6.0
Lombok	Administrada por Spring Boot 3.3.5
Microsoft SQL Server	2025 (RTM), versión 17.0.1000.7

3. Arquitectura

La aplicación sigue una arquitectura en capas. Cada petición recorre las mismas etapas, lo que facilita localizar un error y cambiar una capa sin afectar a las demás.

```
Cliente (frontend / Swagger UI)
        │  JSON + Bearer token
        ▼
Controller   recibe la petición, valida el cuerpo (@Valid) y delega
        │
        ▼
Service      reglas de negocio, transacciones, control de acceso
        │
        ▼
Repository   consultas con Spring Data JPA
        │
        ▼
Model        entidades JPA que se guardan en SQL Server
```

- **DTO como record:** las entradas y salidas son record del paquete dto. Las entidades nunca salen directamente a la API.
- **Seguridad sin estado:** cada petición lleva un token JWT en la cabecera Authorization: Bearer <token>. El filtro JwtAuthenticationFilter lo valida y carga el usuario.
- **Permisos por rol:** se controlan con @PreAuthorize en los controladores y con reglas de dueño en los servicios (por ejemplo, un huésped solo ve sus reservas).
- **Errores uniformes:** GlobalExceptionHandler convierte cualquier error en el mismo formato JSON (sección 10).
- **Transacciones:** los servicios que modifican datos usan @Transactional. Las consultas usan @Transactional(readOnly = true).

4. Estructura del proyecto

El repositorio tiene dos carpetas principales: backend, con la API en Java, y frontend, con la interfaz web. El código del backend se organiza por responsabilidades:

```
backend/
└── src/main/java/sv/edu/udb/sistemahotel/
    ├── SistemaHotelApplication.java
    ├── config/          SecurityConfig, OpenApiConfig, DataInitializer
    ├── controller/      Auth, Perfil, Hotel, Sucursal, TipoHabitacion, Servicio, PoliticaCancelacion,
    │                    Habitacion, Tarifa, Usuario, Huesped, Disponibilidad, Reserva,
    │                    ReservaAcciones, Pago, Reclamo, Consultas
    ├── dto/             records de entrada (*Request) y salida (*Response)
    ├── exception/       ApiException, ErrorResponse, GlobalExceptionHandler
    ├── model/           entidades JPA
    │   └── enums/       EstadoGeneral, EstadoHabitacion, EstadoReserva, EstadoPago, ...
    ├── repository/      interfaces JpaRepository
    ├── security/        JwtService, JwtAuthenticationFilter, UsuarioDetailsService, manejadores 401/403
    └── service/         lógica de negocio

frontend/
├── index.html
├── css/styles.css
├── js/                  config, api, ui, app y las vistas (disponibilidad, reservas, perfil, etc.)
└── assets/              logos y fotografías de habitaciones
```

5. Configuración y ejecución

5.1 Requisitos

- JDK 17 (en IntelliJ: Project SDK y Language level en 17).
- Microsoft SQL Server con el puerto 1433 disponible.
- IntelliJ IDEA con Maven.
- SQL Server Management Studio para crear la base y ejecutar el índice filtrado.

5.2 Base de datos

Antes de ejecutar la aplicación, cree la base de datos:

```
CREATE DATABASE HotelReserva;
```

Las tablas las crea Hibernate al arrancar (spring.jpa.hibernate.ddl-auto=update). La única excepción es un índice filtrado que garantiza un solo titular por reserva, que se crea una vez:

```
USE HotelReserva;
CREATE UNIQUE INDEX ux_reserva_titular ON dbo.reserva_huesped (id_reserva) WHERE es_titular = 1;
```

5.3 Variables de entorno

Las credenciales no van en el código. Se definen como variables de entorno antes de ejecutar la aplicación.

Variable	Obligatoria	Descripción
DB_PASSWORD	Sí	Contraseña de SQL Server.
DB_USER	No (por defecto sa)	Usuario de SQL Server.
JWT_SECRET	Sí	Clave para firmar los tokens. Mínimo 32 caracteres.

En application.properties se usan así:

```
spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=HotelReserva;encrypt=true;trustServer
Certificate=true
spring.datasource.username=${DB_USER:sa}
spring.datasource.password=${DB_PASSWORD}
app.jwt.secret=${JWT_SECRET}
app.jwt.expiration-ms=86400000
```

En IntelliJ IDEA se configuran en Run → Edit Configurations → Environment variables.

5.4 Ejecución

- Ejecutar la clase SistemaHotelApplication.
- La API queda disponible en http://localhost:8080.
- La documentación interactiva está en http://localhost:8080/swagger-ui.html.
- Al arrancar, si la tabla usuario está vacía, DataInitializer crea los roles, los hoteles de ejemplo y los usuarios de prueba (sección 13).
- Para usar el frontend, abrir index.html en el navegador o con un servidor local.

6. Modelo de datos

La base de datos tiene 17 tablas. Las restricciones se definen en las entidades (@Check, unique, nullable) y las crea Hibernate. La figura 1 muestra el modelo relacional implementado.

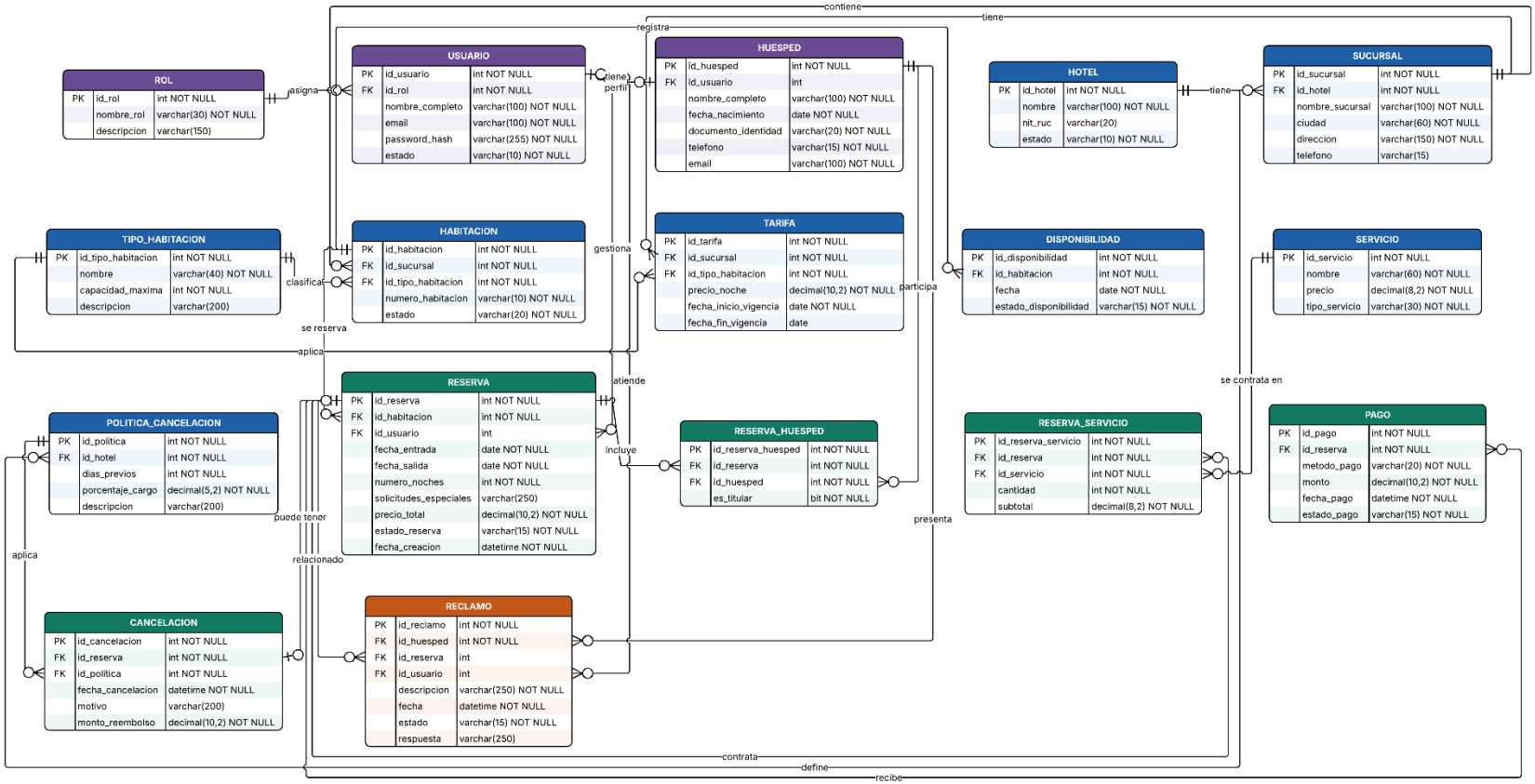


***Figura 1**. Diagrama entidad-relación del modelo implementado. Debe coincidir con la base de datos creada por Hibernate*

Tabla	Propósito	Restricciones principales
hotel	Hoteles	nit_ruc único (permite nulo)
sucursal	Sucursales de cada hotel	FK a hotel
tipo_habitacion	Tipos (INDIVIDUAL, DOBLE, SUITE, FAMILIAR)	capacidad_maxima entre 1 y 10
habitacion	Habitaciones de una sucursal	numero_habitacion único por sucursal
tarifa	Precio por noche por sucursal y tipo, con vigencia	precio_noche > 0; fin de vigencia ≥ inicio
disponibilidad	Días bloqueados u ocupados por habitación	Único por habitación y fecha
rol	ADMIN, RECEPCIONISTA, HUESPED	nombre_rol único
usuario	Cuentas de acceso	email único
huesped	Datos de personas que se hospedan	documento_identidad único; id_usuario puede ser nulo
servicio	Servicios adicionales	precio ≥ 0
reserva	Reserva de una habitación	fecha_salida > fecha_entrada; numero_noches ≥ 1
reserva_huesped	Huéspedes de una reserva	es_titular (un solo titular por reserva)
reserva_servicio	Servicios contratados en una reserva	cantidad ≥ 1; subtotal ≥ 0
pago	Pagos de una reserva	monto > 0
politica_cancelacion	Reglas de cargo por cancelación, por hotel	dias_previos ≥ 0; porcentaje_cargo entre 0 y 100
cancelacion	Cancelación de una reserva (una por reserva)	monto_reembolso ≥ 0
reclamo	Reclamos de huéspedes	Estado ABIERTO, EN_PROCESO o CERRADO

7. Roles y permisos

Acción	HUESPED	RECEPCIONISTA	ADMIN
Login y registro	Sí	Sí	Sí
Ver catálogos (hoteles, sucursales, tarifas, etc.)	Sí	Sí	Sí
Crear, editar o eliminar catálogos	No	No	Sí
Buscar disponibilidad	Sí	Sí	Sí
Crear reserva	Sí, a su nombre	Sí	Sí
Ver reservas	Solo las suyas	Todas	Todas
Pagar, cancelar y cambiar fechas	Solo las suyas	Sí	Sí
Finalizar reserva	No	Sí	Sí
Crear reclamo	Solo con su huésped	Sí	Sí
Atender reclamo	No	Sí	Sí
Administrar huéspedes	No	Sí	Sí
Administrar usuarios	No	No	Sí
Pagos y estadísticas	No	Sí	Sí

8. Referencia de los endpoints

Todos los endpoints requieren token, excepto los de autenticación. La documentación completa, con los esquemas de entrada y salida, está en Swagger UI (http://localhost:8080/swagger-ui.html). Para probar un endpoint protegido, use el botón Authorize con el token obtenido en /api/auth/login.

8.1 Autenticación y perfil

Método	Ruta	Descripción
POST	`/api/auth/login`	Devuelve el token JWT, su tipo, su expiración y los datos del usuario.
POST	`/api/auth/registro`	Crea una cuenta de HUESPED y su perfil de huésped.
GET	`/api/auth/me`	Datos del usuario autenticado (id, nombre, correo y rol).
GET	`/api/perfil`	Datos personales del huésped autenticado (nombre, documento, fecha de nacimiento, teléfono). Responde 404 si la cuenta no tiene perfil de huésped, como ocurre con el personal.

8.2 Catálogos

Método	Ruta	Descripción
GET / POST	`/api/hoteles`	Listar y crear hoteles.
GET / PUT / DELETE	`/api/hoteles/{id}`	Ver, editar y eliminar hotel (409 si tiene sucursales).
GET / POST	`/api/sucursales`	Listar (filtro hotelId) y crear.
GET / PUT / DELETE	`/api/sucursales/{id}`	Ver, editar y eliminar (409 si tiene habitaciones o tarifas).
GET / POST	`/api/tipos-habitacion`	Listar y crear tipos.
GET / PUT / DELETE	`/api/tipos-habitacion/{id}`	Ver, editar y eliminar (409 si está en uso).
GET / POST	`/api/habitaciones`	Listar (filtro sucursalId) y crear.
GET / PUT / DELETE	`/api/habitaciones/{id}`	Ver, editar y eliminar.
GET / POST	`/api/tarifas`	Listar (filtros sucursalId, tipoHabitacionId) y crear.
GET / PUT / DELETE	`/api/tarifas/{id}`	Ver, editar y eliminar.
GET / POST	`/api/servicios`	Listar y crear servicios.
GET / PUT / DELETE	`/api/servicios/{id}`	Ver, editar y eliminar.
GET / POST	`/api/politicas-cancelacion`	Listar (filtro hotelId) y crear.
GET / PUT / DELETE	`/api/politicas-cancelacion/{id}`	Ver, editar y eliminar.

8.3 Usuarios y huéspedes

Método	Ruta	Permiso	Descripción
GET / POST	`/api/usuarios`	ADMIN	Listar y crear personal (ADMIN o RECEPCIONISTA).
PUT / DELETE	`/api/usuarios/{id}`	ADMIN	Editar y eliminar (409 si tiene reservas, reclamos o perfil de huésped).
PATCH	`/api/usuarios/{id}/activar`	ADMIN	Activa la cuenta.
PATCH	`/api/usuarios/{id}/desactivar`	ADMIN	Desactiva la cuenta (no permite desactivar la propia).
GET / POST	`/api/huespedes`	ADMIN, RECEPCIONISTA	Listar y crear huéspedes.
GET / PUT / DELETE	`/api/huespedes/{id}`	ADMIN, RECEPCIONISTA	Ver, editar y eliminar (409 si tiene reservas o reclamos). Al eliminar, la cuenta asociada queda INACTIVA.

8.4 Disponibilidad, reservas y pagos

Método	Ruta	Descripción
GET	`/api/disponibilidad`	Habitaciones libres. Parámetros: fechaEntrada, fechaSalida, huespedes, y opcionales ciudad y tipoHabitacionId.
GET / POST	`/api/reservas`	Listar (filtros estado, desde, hasta) y crear reserva.
GET	`/api/reservas/{id}`	Ver reserva.
POST	`/api/reservas/{id}/pagos`	Registrar pago simulado.
POST	`/api/reservas/{id}/cancelar`	Cancelar con cálculo de cargo y reembolso.
PUT	`/api/reservas/{id}/fechas`	Cambiar fechas y recalcular precio.
PATCH	`/api/reservas/{id}/finalizar`	Finalizar reserva (ADMIN, RECEPCIONISTA).
GET	`/api/pagos`	Listado de pagos por fecha (ADMIN, RECEPCIONISTA).

8.5 Reclamos y estadísticas

Método	Ruta	Descripción
GET / POST	`/api/reclamos`	Listar (filtro estado) y crear reclamo.
PATCH	`/api/reclamos/{id}/atender`	Atender reclamo (ADMIN, RECEPCIONISTA).
GET	`/api/estadisticas`	Indicadores del rango desde–hasta (ADMIN, RECEPCIONISTA).

9. Reglas de negocio

9.1 Reservas

- La fecha de entrada no puede ser anterior a hoy, y la de salida debe ser posterior a la de entrada.
- La habitación debe estar DISPONIBLE, no tener reservas PENDIENTE o CONFIRMADA que se crucen con el rango, y no tener días BLOQUEADA u OCUPADA.
- La capacidad incluye al titular: titular más huéspedes adicionales no puede pasar de la capacidad máxima del tipo.
- El precio es precio_noche × noches + Σ(precio del servicio × cantidad), con la tarifa vigente el día de entrada.
- Una reserva nueva queda PENDIENTE. Cuando un pago cubre todo el saldo, pasa a CONFIRMADA.
- Solo una reserva CONFIRMADA puede pasar a FINALIZADA, y solo si el saldo es cero.
- Al cambiar fechas se revalida la disponibilidad excluyendo la propia reserva, y se recalcula el precio.
- Un hotel INACTIVO no aparece en la búsqueda y no acepta reservas nuevas.

9.5 Tarifas

- Dos tarifas del mismo tipo y sucursal no pueden tener fechas que se crucen. Si se cruzan, la API responde 409.

9.2 Pagos

- El monto debe ser mayor a 0, no mayor al saldo pendiente, y tener como máximo dos decimales.
- Solo se pagan reservas PENDIENTE o CONFIRMADA.
- El pago es simulado: no hay pasarela de pago y el sistema no guarda datos de tarjetas ni de cuentas bancarias.

9.3 Cancelaciones

- Solo se cancelan reservas PENDIENTE o CONFIRMADA cuya fecha de entrada sea hoy o posterior.
- Días restantes = fecha de entrada − hoy. Se aplica la política del hotel con el mayor dias_previos que sea menor o igual a los días restantes. Si no hay ninguna, el cargo es del 100 %.
- Cargo = precio total × porcentaje ÷ 100. Reembolso = lo pagado − cargo, sin bajar de 0.

9.4 Estadísticas

- Se cuentan las reservas cuya fecha de entrada cae dentro del rango.
- Noches ocupadas: reservas CONFIRMADA y FINALIZADA.
- Noches disponibles: número de habitaciones × días del rango, contando ambos extremos.
- Ocupación = noches ocupadas ÷ noches disponibles × 100.
- Ingreso neto = pagos completados − reembolsos.

10. Formato de errores

Todas las respuestas de error tienen la misma forma, para que el frontend pueda mostrar mensajes claros sin importar el módulo que falló:

```
{
  "timestamp": "2026-11-01T10:15:30",
  "status": 409,
  "error": "Conflicto",
  "mensaje": "La habitación no está disponible para las fechas indicadas.",
  "detalles": []
}
```

Código	Cuándo ocurre
400	Datos inválidos, formato incorrecto o regla de negocio no cumplida.
401	Falta el token, token inválido o credenciales incorrectas.
403	El usuario no tiene el rol necesario.
404	El recurso no existe o pertenece a otro usuario.
409	Conflicto: duplicados, dependencias o estado no permitido.
500	Error inesperado del servidor (se registra en el log).

11. Pruebas del sistema

Las pruebas se realizaron de forma manual en tres niveles: los endpoints desde Swagger UI, con cada uno de los tres roles; el flujo completo desde el frontend (búsqueda, reserva, pago, confirmación y perfil); y la revisión de los datos en SQL Server después de cada operación.

Algunas validaciones se comprobaron a propósito con datos incorrectos, por ejemplo, finalizar una reserva que no está CONFIRMADA (debe responder 409) o consultar /api/perfil con una cuenta de personal (debe responder 404).

12. Limitaciones y trabajo futuro

- El pago es simulado. No hay integración con pasarelas de pago.
- No hay paginación en los listados. Con muchos registros convendría añadirla.
- La disponibilidad se calcula en memoria a partir de las habitaciones. Con muchas habitaciones conviene consultar solo las necesarias.
- El token JWT dura 24 horas y no hay refresh token.
- No hay pruebas automatizadas (JUnit). La verificación se hizo desde Swagger UI, el frontend y consultas SQL.
- No se envían correos de confirmación.

13. Usuarios y datos de prueba

Los usuarios siguientes se crean solo si la tabla usuario está vacía. Son para entorno de desarrollo y no deben usarse en producción.

Usuario	Correo	Contraseña	Rol
Administrador	admin@hotel.sv	Admin123*	ADMIN
Recepcionista	recepcion@hotel.sv	Recep123*	RECEPCIONISTA
Huésped	huesped@hotel.sv	Huesped123*	HUESPED

13.1 Datos base

- **Hoteles:** tres (Plaza Central, Marea Azul y Cumbre Verde), cada uno con sus propias políticas de cancelación.
- **Sucursales:** siete, en San Salvador, Santa Tecla, Santa Ana, Sonsonate, La Libertad, San Miguel y Usulután.
- **Tipos de habitación:** cuatro (INDIVIDUAL, DOBLE, SUITE y FAMILIAR).
- **Habitaciones:** 28 en total. Dos de ellas están en MANTENIMIENTO para verificar que no aparecen en la búsqueda.
- **Tarifas:** para cada sucursal y tipo hay tres periodos: temporada normal de 2026 (hasta el 30 de noviembre); temporada alta de fin de año (del 1 de diciembre de 2026 al 15 de enero de 2027, con un 30 % más); y temporada normal de 2027 (desde el 16 de enero, con un 5 % más). El precio base cambia según la sucursal.
- **Servicios adicionales:** cinco.
- **Políticas de cancelación:** tres por hotel: 7 días o más sin cargo (0%), de 2 a 6 días con 50%, y menos de 2 días con 100%.

14. Repositorio y acceso

Concepto	Detalle
URL del repositorio	https://github.com/Manu-Hdez29/sistema-hotel (público)
Colaboradores del repositorio	Manuel Miranda y Fabricio Salazar.
Entrega	El último commit debe quedar subido antes del 29/10/2026 a las 23:59. Después de esa hora no se hacen cambios en el código.